

POWERSHELL OBFUSCATION AND DETECTION

SOC Analyst Lab Project

Lab Date: 24 Mach 2026

Report Date: 24 March 2026

Project by: Mohammad Mubashir

Environment: VirtualBox Lab

Tools Used: Windows 11, PowerShell, Event Viewer

1. INTRODUCTION

1.1 Project Objective

The main objective of this project is to detect obfuscated PowerShell scripts used by attackers to bypass security controls. Obfuscation techniques like Base64 encoding are commonly used to hide malicious commands. This project demonstrates how PowerShell Script Block Logging can capture and decode such obfuscated scripts for forensic analysis.

1.2 Scope

This project covers:

- Enabling PowerShell Script Block Logging
- Creating a plain PowerShell script
- Obfuscating the script using Base64 encoding
- Executing the obfuscated script
- Detecting the decoded script in Event Logs (Event ID 4104)
- Documenting Indicators of Compromise (IOCs)

1.3 Tools Used

Tool	Purpose
Windows 11	Local machine for execution
PowerShell	Script execution and logging
Event Viewer	Log analysis (Event ID 4104)
Base64 Encoding	Obfuscation technique

2. LAB SETUP

2.1 Lab Environment

Machine	Operating System	Role
Victim / Analyst	Windows 11	PowerShell execution and log analysis

2.2 Environment Description

This project is executed on a single Windows 11 machine. No network connectivity is required as all operations are performed locally. The focus is on PowerShell logging and event analysis.

2.3 PowerShell Logging Configuration

PowerShell Script Block Logging was enabled to capture all executed PowerShell commands. This setting is configured via Windows Registry and logs events under Event ID 4104.

3. ENABLE SCRIPT BLOCK LOGGING

3.1 Create Registry Key for Enable Script Block Logging

```
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell" -Name "ScriptBlockLogging" -Force
```

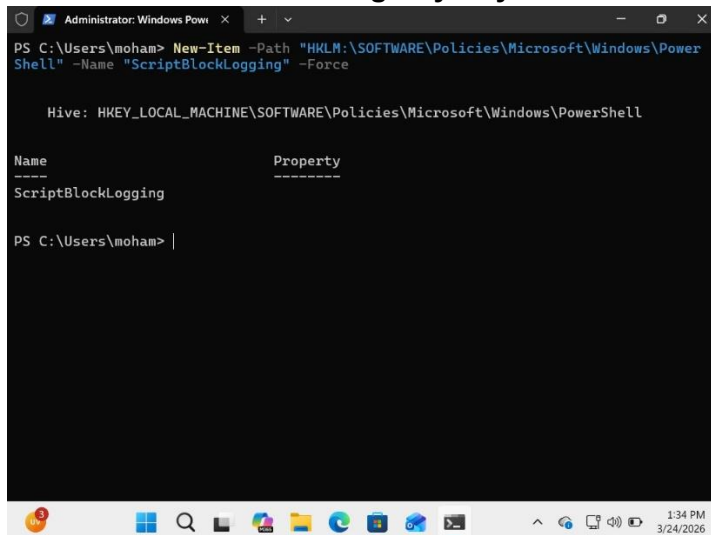
Explanation:

- New-Item → Creates new registry key
- -Path → Parent registry location
- -Name → New key name
- -Force → Creates even if parent path doesn't exist

Why?

The registry path may not exist by default. This command creates it before enabling logging.

Screenshot 01 - Create Registry Key



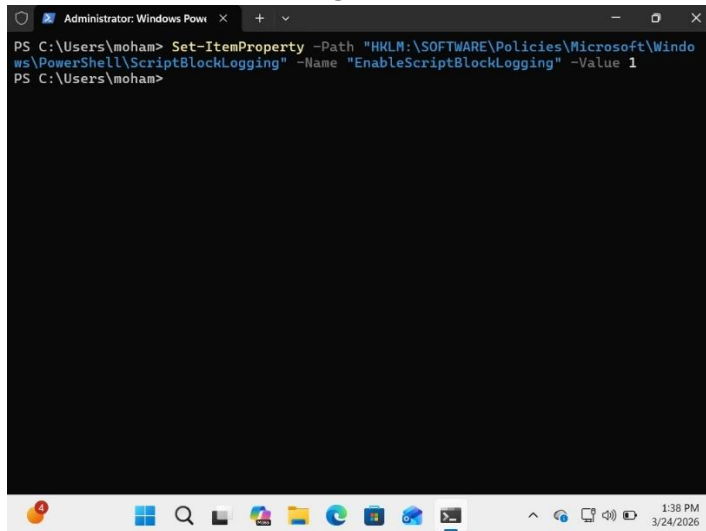
3.2 Enable Script Block Logging

```
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Name "EnableScriptBlockLogging" -Value 1
```

Explanation:

- Set-ItemProperty → Modifies registry value
- -Path → Registry location
- -Name → Setting name
- -Value 1 → Enable logging

Screenshot 2 – Enabling PowerShell Script Block Logging



```
Administrator: Windows Pow
PS C:\Users\moham> Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Name "EnableScriptBlockLogging" -Value 1
PS C:\Users\moham>
```

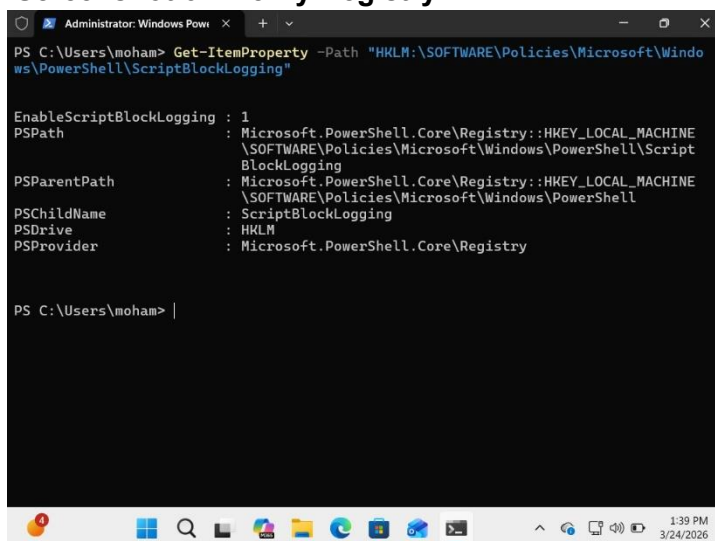
3.3 Verify Logging is Enabled

Get-ItemProperty -Path
"HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"

Explanation:

- Get-ItemProperty → Reads registry values
- Output shows EnableScriptBlockLogging = 1 if enabled

Screenshot 3 – Verify Registry



```
Administrator: Windows Pow
PS C:\Users\moham> Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"

EnableScriptBlockLogging : 1
PSPath                   : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging
PSParentPath              : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\PowerShell
PSChildName               : ScriptBlockLogging
PSDrive                   : HKLM
PSProvider                : Microsoft.PowerShell.Core\Registry

PS C:\Users\moham> |
```

4. CREATE OBFUSCATED POWERSHELL SCRIPT

4.1 Create Plain Script

```
@  
Write-Host "Hello from PowerShell"  
Start-Process notepad.exe  
"@ | Out-File -FilePath C:\Users\Public\plain-script.ps1
```

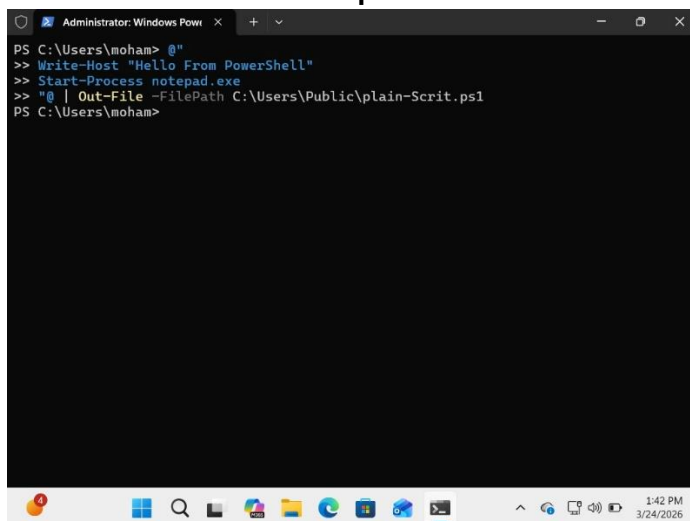
Explanation:

- @" ... "@ → Multi-line string
- Out-File → Saves content to file
- -FilePath → Location to save

Why?

First create a plain script that we will later obfuscate

Screenshot 4 – Plain Script Created



4.2 Base64 Encode the Script

```
$command = 'Write-Host "Hello from PowerShell"; Start-Process notepad.exe'  
$bytes = [System.Text.Encoding]::Unicode.GetBytes($command)  
$encoded = [Convert]::ToBase64String($bytes)  
Write-Host $encoded
```

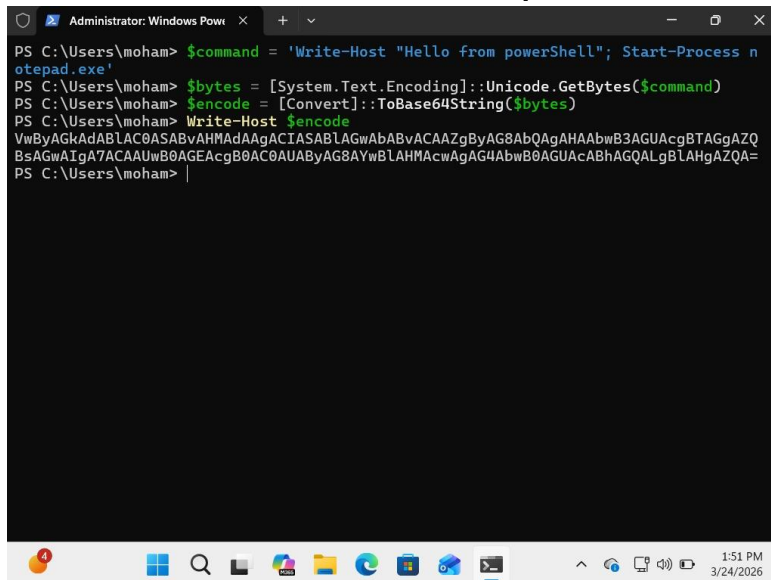
Explanation:

- \$command → Stores the script
- GetBytes() → Converts string to bytes
- ToBase64String() → Encodes bytes to Base64
- Write-Host → Displays the encoded string

Why?

Base64 encoding is a common obfuscation technique used by attackers to hide malicious commands.

Screenshot 5 – Base64 Encoded Script



```
PS C:\Users\moham> $command = 'Write-Host "Hello from powerShell"; Start-Process notepad.exe'
PS C:\Users\moham> $bytes = [System.Text.Encoding]::Unicode.GetBytes($command)
PS C:\Users\moham> $encode = [Convert]::ToBase64String($bytes)
PS C:\Users\moham> Write-Host $encode
VwByAGkAdABlAC0ASABvAHMAdAAgACIASABlAGwAbABvACAAZgByAG8AbQAgAHAAbwB3AGUAcgBTAGgAZQ
BsAGwAIgA7ACAAUwB0AGEAcgB0AC0AUABYAG8AYwBlAHMAcwAgAG4AbwB0AGUAcABhAGQALgBlAHgAZQA=
PS C:\Users\moham>
```

4.3 Execute Obfuscated Script

powershell.exe -EncodedCommand [encoded string show in Screenshot 5 and 6

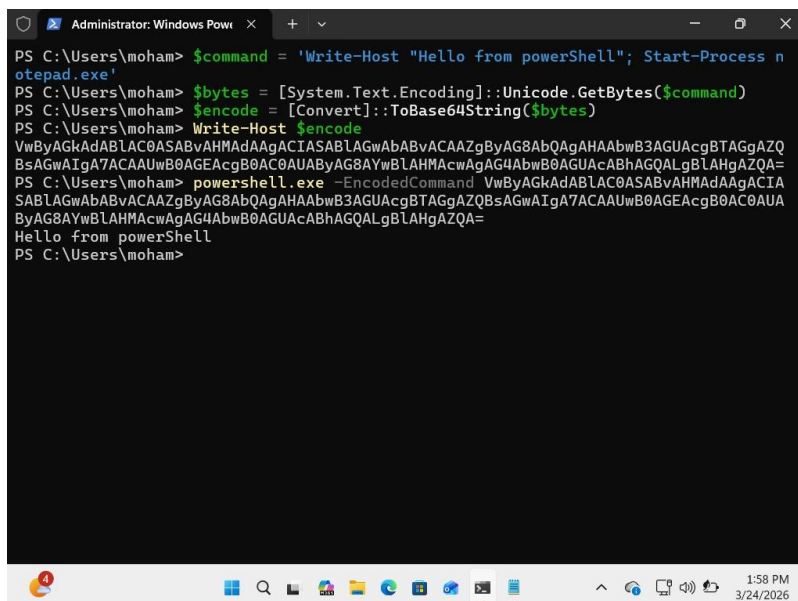
Explanation:

- -EncodedCommand → Accepts Base64 encoded command
- Executes without showing plain text

Why?

Attackers use this to bypass simple string-based detection.

Screenshot 6A – Obfuscated Command Run



```
PS C:\Users\moham> $command = 'Write-Host "Hello from powerShell"; Start-Process notepad.exe'
PS C:\Users\moham> $bytes = [System.Text.Encoding]::Unicode.GetBytes($command)
PS C:\Users\moham> $encode = [Convert]::ToBase64String($bytes)
PS C:\Users\moham> Write-Host $encode
VwByAGkAdABlAC0ASABvAHMAdAAgACIASABlAGwAbABvACAAZgByAG8AbQAgAHAAbwB3AGUAcgBTAGgAZQ
BsAGwAIgA7ACAAUwB0AGEAcgB0AC0AUABYAG8AYwBlAHMAcwAgAG4AbwB0AGUAcABhAGQALgBlAHgAZQA=
PS C:\Users\moham> powershell.exe -EncodedCommand VwByAGkAdABlAC0ASABvAHMAdAAgACIA
SABlAGwAbABvACAAZgByAG8AbQAgAHAAbwB3AGUAcgBTAGgAZQBzAGwAIgA7ACAAUwB0AGEAcgB0AC0AU
ByAG8AYwBlAHMAcwAgAG4AbwB0AGUAcABhAGQALgBlAHgAZQA=
Hello from powerShell
PS C:\Users\moham>
```

[illegible]

5.1 Event Viewer – Filter Event ID 4104

1. Open Event Viewer (eventvwr.msc)
2. Navigate to: Applications and Services Logs → Microsoft → Windows → PowerShell → Operational
3. Click Filter Current Log
4. Enter Event ID: 4104
5. Click OK

Event ID 4104 (Script Block Logging) captures every PowerShell command executed, including obfuscated ones.

Event Viewer

File Action View Help

Operational Number of events: 416

Filtered Log: Microsoft-Windows-PowerShell/Operational; Source; Event ID:

Level	Date and Time	Source	Event ID	Task Cat...
Verbose	3/24/2025 1:58:03 PM	PowerS...	4104	Execute ...
Verbose	3/24/2025 1:58:01 PM	PowerS...	4104	Execute ...
Verbose	3/24/2025 1:45:32 PM	PowerS...	4104	Execute ...
Verbose	3/24/2025 1:45:19 PM	PowerS...	4104	Execute ...
Verbose	3/24/2025 1:45:18 PM	PowerS...	4104	Execute ...
Verbose	3/24/2025 1:45:11 PM	PowerS...	4104	Execute ...
Warning	3/24/2025 1:39:04 PM	PowerS...	4104	Execute ...
Warning	3/24/2025 1:38:07 PM	PowerS...	4104	Execute ...
Warning	3/24/2025 1:34:13 PM	PowerS...	4104	Execute ...
Warning	3/24/2025 1:33:55 PM	PowerS...	4104	Execute ...
Warning	3/24/2025 1:30:59 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...
Warning	3/14/2025 3:10:15 PM	PowerS...	4104	Execute ...

Clears events from log.

Actions

Operational

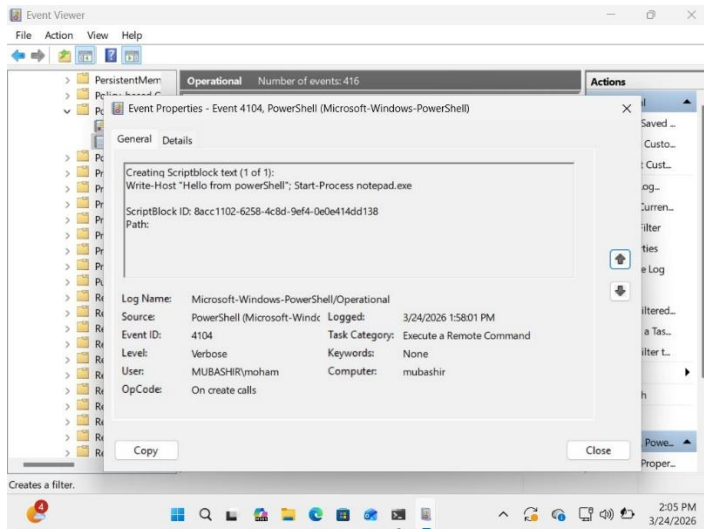
- Open Saved ...
- Create Custo...
- Import Cust...
- Clear Log...
- Filter Current...
- Clear Filter
- Properties
- Disable Log
- Find...
- Save Filtered...
- Attach a Tass...
- Save Filter L...
- View
- Refresh
- Help
- Event 4104, Powe...
- Event Proper...

5.2 Decoded Script in Event Details

Observation:

When an obfuscated script is executed, Event ID 4104 logs the decoded version of the script in the Message section.

Screenshot 7B – Decoded Script in Event



5.3 Key Detection Findings

- Script Block Logging successfully captured the obfuscated command
- Event ID 4104 displayed the original (decoded) script
- Base64 encoding did NOT bypass PowerShell logging
- SOC analysts can detect obfuscated attacks using this method

6. INDICATORS OF COMPROMISE

6.1 IOCs Identified in This Project

IOC NO.	Category	Indicator	Description
IOC 01	Event log	Event ID 4104	Triggered with encoded command Indicates obfuscated script execution attempt
IOC 02	File System	plain-script.ps1 in C:\Users\Public	Suspicious PowerShell script location
IOC 03	Obfuscation	Base64 encoded command	Attackers use Base64 to hide malicious intent
IOC 04	Process Execution	powershell.exe - EncodedCommand	Execution of obfuscated PowerShell command

IOC 05	Process Chain	PowerShell spawning Notepad	Unusual parent-child process relationship
IOC 06	Log Evidence	Event ID 4104 with decoded script	Forensic evidence of obfuscated script execution

6.2 IOC Severity Levels

IOC NO.	Severity	Justification
IOC 01	Critical	Event ID 4104 Triggered with encoded command Indicates obfuscated script execution attempt
IOC 02	Medium	Script in public folder – unusual location
IOC 03	Critical	Base64 encoding indicates obfuscation attempt
IOC 04	Critical	Direct indicator of malicious execution
IOC 05	High	Process chain deviation (Word → PowerShell → Notepad)
IOC 06	Critical	Forensic evidence of attack

6.3 Detection Rules

PowerShell Logging Rule:

Event ID: 4104

Condition: Script Block contains encoded command patterns

Action: Alert SOC for investigation

Process Monitoring Rule:

Process: powershell.exe

Parent: winword.exe / excel.exe / outlook.exe

Action: High severity alert

6.4 Importance of IOCs in SOC

In a Security Operations Center (SOC), IOCs are used to:

- Detect ongoing or past intrusions
- Correlate events across multiple data sources
- Trigger alerts for immediate response
- Improve threat hunting and forensic analysis
- Enhance security monitoring rules and use cases

7. CONCLUSION

7.1 Project Summary

This project successfully demonstrated how PowerShell Script Block Logging can detect obfuscated scripts.

A plain PowerShell script was created, obfuscated using Base64 encoding, and executed. Event ID 4104 captured and decoded the script, proving that even obfuscated commands can be detected.

7.2 Key Achievements

- Enabled PowerShell Script Block Logging
- Created and obfuscated a PowerShell script using Base64
- Executed obfuscated script
- Detected decoded script in Event ID 4104
- Identified 6 key IOCs
- Documented detection rules for SOC

7.3 Learning Outcomes

Skill	What I Learned
PowerShell Security	How attackers use obfuscation to bypass defences
Logging & Detection	How Script Block Logging captures decoded commands
Forensic Analysis	How to investigate Event ID 4104
IOC Identification	How to document obfuscation indicators
SOC Monitoring	How to create detection rules for obfuscated scripts

7.4 Challenges Faced & Solutions

Challenge	Solution
Registry path didn't exist	Created key using New-Item -Force
Event logs empty	Verified logging was enabled before execution

7.5 Future Scope

This project can be extended by:

- Using Sysmon to track process creation (Event ID 1)
- Creating SIEM alerts for repeated obfuscated commands
- Analyzing other obfuscation techniques (XOR, GZip)
- Setting up AMSI (Antimalware Scan Interface) for real-time detection
- Simulating real attacker scenarios with C2 frameworks

8. PROJECT DETAILS

Field	Information
Author	Mohammad Mubashir
Date	24 March 2026
Environment	Windows 11 (Local Machine)
Project	PowerShell Obfuscation Detection
Tools Used	Windows 11, PowerShell, Event Viewer

9. GITHUB REPOSITORY

<https://github.com/Mubashir-soc-01/SOC-Project>

10. LINKEDIN PROFILE

<https://www.linkedin.com/in/mohammad-mubashir-b729643b7/>

THANK YOU

Project by: Mohammad Mubashir

Date: 24 Mach 2026